

# EduGenie Project Summary Brief

<https://gemini.google.com/app/fc43892a2a24b015>

User prompt: etha konjam brief fa kudu

Response: Indha **EduGenie** project-oda details ellam simple ah, crisp ah concise ah brief-a keela kuduthurken:

## EduGenie – Project Summary

**EduGenie** engiradhu **Google Gemini 1.5 Pro** matrum local AI model (**LaMini-Flan-T5**) combine panni create panna oru lightweight **AI Educational Assistant**. Standard hardware (e.g., Mac M1) la kuda heavy GPU requirement illama idhu run aagum.

## 8 Key Project Phases

### 1. Brainstorming & Ideation

- **Goal:** Self-learning students and teachers-ku dynamic and personalized study assistance tharuvadhu.
- **Core Value:** Cloud AI (Gemini) + Local CPU model mix panni efficient ah run panradhu.

### 2. Requirement Analysis & Tech Stack

- **Tech Stack:** Python 3.10+, FastAPI, Uvicorn, Jinja2, Hugging Face Transformers.
- **AI Models:**
  - Cloud: Google Gemini 1.5 Pro
  - Local CPU: LaMini-Flan-T5-783M (offline explanations-ku)

### 3. System Architecture & Project Structure

Client (HTML/CSS) → FastAPI Backend Router → Specific Modules → AI Models (Gemini / LaMini) → Frontend Display.

#### Key Project Files:

- `main.py` – REST API Routing & Endpoints
- `explanation_module.py` – Local model concept explanation
- `qna.py` – Gemini Q&A
- `quiz_module.py` – MCQ Generation (JSON output)
- `summary_module.py` – Text summarization
- `learning_path.py` – Subject-based roadmap generation

### 4. Project Planning & Scenarios

- **Milestones:** Model selection → Core Module Dev → UI Integration → Local Deployment.
- **Use Cases:** Instant Q&A, Pythagoras Theorem quiz, SQL roadmaps.

### 5. Project Development (API Endpoints)

1. GET `/qa`: Academic questions-ku direct answers.
2. POST `/explain/`: School level-la simple concept explanations (LaMini local CPU).
3. POST `/quiz`: Input text-la irundhu 3 MCQs (JSON format) generate pannum.
4. POST `/summarize/`: Periya text-a short-a condense pannum.
5. GET `/learn/recommendations`: Subject-ku multi-level learning path (Beginner to Advanced) tharum.

### 6. Testing

- Quantum Computing, Binary Search, Industrial Revolution, SQL etc. topics la test panni functional verification mudikapadhadhu.

### 7. Setup & Execution Guide

```
# 1. Dependencies install
pip install fastapi uvicorn jinja2 torch transformers google-generativeai[cite: 1]

# 2. Set API Key
export GEMINI_API_KEY="your_api_key_here"[cite: 1]

# 3. Server start
uvicorn main:app --reload[cite: 1]
```

- **Dashboard Access:** [http://127.0.0.1:8000](http://127.0.0.1:8000)
- **Author / Mentor:** Tella Divya Sree / Siri

## 8. Future Roadmap

- Voice Interaction (STT/TTS)
- Multilingual support
- Mobile App with offline cache
- Student Progress Tracking & LMS/OCR integration

User prompt: *naraiya page vara mathiri documentation create pannitha*

Response: Here is a comprehensive and detailed **Project Documentation** for **EduGenie**, formatted cleanly with all technical modules, code snippets, system architecture, setup steps, and future roadmaps.

You can copy and save this directly into a document or Markdown file.

# Project Documentation: EduGenie

**Project Name:** EduGenie - Google Gemini Powered Learning Assistant

**Author:** Tella Divya Sree

**Mentor:** Siri

**Submission Date:** 11/04/2025

**Repository Structure:** AI-ML and Gen-AI Track Project

## Table of Contents

1. [Executive Summary & Ideation](#)
2. [Requirement Analysis](#)
3. [System Design & Architecture](#)
4. [Project Planning & Development Milestones](#)
5. [Detailed Module Implementation & Source Code](#)
6. [Testing Strategy & Output Verification](#)
7. [Installation, Environment Setup & Deployment](#)
8. [Demonstration & Future Roadmap](#)

## 1. Executive Summary & Ideation

### 1.1 Project Concept

**EduGenie** is an intelligent, lightweight, AI-powered educational assistant designed to streamline self-directed learning. It breaks down complex academic subjects into digestible concepts, generates interactive assessments, summarizes extensive passages, and recommends personalized learning paths.

### 1.2 Problem Statement

Traditional educational content is often static, overly dense, or difficult to digest for self-directed learners without dedicated tutors. Students frequently encounter friction when trying to get instant answers, test their knowledge with contextual quizzes, or find structured step-by-step guides for new domains.

1.3 Target Audience

- **K-12 & Higher Education Students:** For homework assistance, exam preparation, and conceptual clarity.
- **Self-Learners & Professionals:** For mastering new technical subjects through structured roadmaps.
- **Educators & Content Creators:** For automated generation of multiple-choice practice quizzes and concise summaries.

1.4 Core Value Proposition

- **Hybrid AI Architecture:** Combines high-performance cloud AI engines (**Google Gemini 1.5 Pro**) with specialized local open-source models (**LaMini-Flan-T5-783M**).
- **Hardware Efficiency:** Designed to operate seamlessly on standard consumer laptops (such as Apple Silicon Mac M1) without requiring expensive dedicated cloud GPU clusters.
- **Modularity:** Fully decoupled micro-modules built with Python and FastAPI for fast iteration and clear separation of concerns.

2. Requirement Analysis

2.1 Functional Requirements

| Module / Endpoint      | Request Method | Primary Function                                                  | Input                   | Output                                |
|------------------------|----------------|-------------------------------------------------------------------|-------------------------|---------------------------------------|
| /qa                    | GET            | Smart direct answering of academic & general knowledge questions. | Query string (question) | Concise factual answer                |
| /explain/              | POST           | Simplified breakdown of complex topics.                           | JSON { "topic": "..." } | School-friendly explanation           |
| /quiz                  | POST           | Automated multiple-choice question generation.                    | JSON { "text": "..." }  | Clean JSON array of 3 MCQs            |
| /summarize/            | POST           | Key information extraction from long text.                        | JSON { "text": "..." }  | Concise summary                       |
| /learn/recommendations | GET            | Multi-stage topic roadmap creation.                               | Query string (topic)    | Adaptive learning path with resources |

2.2 Non-Functional Requirements

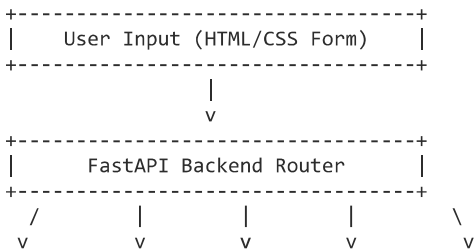
1. **Low Latency & High Throughput:** Response times kept under 2–3 seconds for general queries using lightweight routing.
2. **Error Resilience & Fallbacks:** Automated JSON cleanups for raw AI responses and API error handling.
3. **Extensibility:** Standard REST interface allowing easy integration with web, mobile, or voice interfaces.

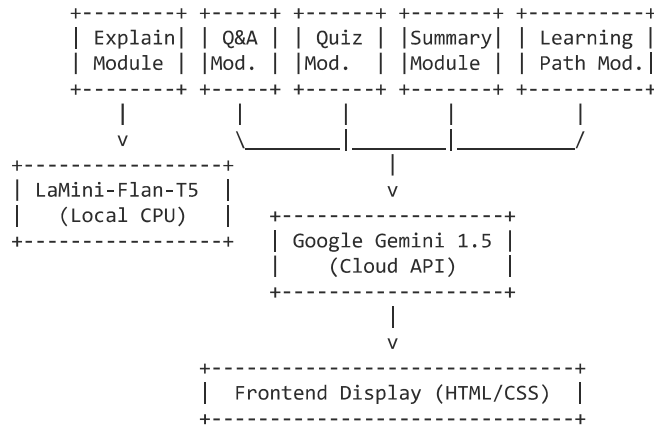
2.3 Technical Dependencies

- **Programming Language:** Python 3.10+
- **Web Framework:** FastAPI + Uvicorn (ASGI)
- **Frontend Templating:** Jinja2 (HTML5 / CSS3)
- **Cloud AI API:** google-generativeai (Gemini 1.5 Pro)
- **Local Machine Learning Framework:** transformers, torch (MBZUAI/LaMini-Flan-T5-783M)

3. System Design & Architecture

3.1 Flowchart Diagram





```
```[cite: 1]
```

### 3.2 Repository Directory Structure

```
```text
EduGenie/
├── main.py                # Central FastAPI REST server and routing
├── explanation_module.py  # Concept explanation logic using LaMini-Flan-T5
├── qna.py                 # Question-answering pipeline via Gemini 1.5 Pro
├── quiz_module.py         # Quiz generator & JSON string cleaner
├── summary_module.py      # Passage summarizer logic
├── learning_path.py       # Structured roadmap generation logic
├── requirements.txt       # Project environment dependencies
├── templates/
│   └── index.html         # HTML5 User Interface dashboard
├── static/
│   └── style.css          # Custom layout styles and response styling
```[cite: 1]
```

```
---
```

## 4. Project Planning & Development Milestones

### 4.1 Milestone Breakdown

```

* **Milestone 1: Model Selection & Architecture Design**
  Evaluated cloud vs local model constraints. Selected Google Gemini 1.5 Pro for cloud API tasks and LaMini-Flan-T5-783M

* **Milestone 2: Core Logic Implementation**
  Created standalone Python modules for Q&A, Summarization, Quiz Generation, Concept Explanation, and Roadmap Generation

* **Milestone 3: Web Integration & API Routing**
  Developed FastAPI endpoints connecting web payload calls to model inference pipelines[cite: 1].

* **Milestone 4: End-to-End Testing & Optimization**
  Performed validation across all features, clean JSON response extraction, and fallback error handling[cite: 1].
  
```

### 4.2 Target User Scenarios

```

* **Scenario A (Instant Q&A):** A student enters *"Which is the largest ocean?"* into the query box and immediately rece
* **Scenario B (Interactive Assessment):** A student pastes a chapter about the *Pythagoras Theorem* and receives 3 form
* **Scenario C (Structured Study):** A learner searching for *SQL* receives a structured beginner-to-advanced roadmap co
  
```

```
---
```

## 5. Detailed Module Implementation & Source Code

### 5.1 Concept Explanation Module (`explanation\_module.py`)

Executes locally using Hugging Face Transformers without relying on cloud credits for basic explanations[cite: 1].

```
```python
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load lightweight local model
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")
  
```

```

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```[cite: 1]

---

### 5.2 Question Answering Module (`qna.py`)
Uses Google Gemini 1.5 Pro for broad domain knowledge answering[cite: 1].

```python
import google.generativeai as genai

def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"Error in QnA: {e}"
```[cite: 1]

---

### 5.3 Quiz Generation Module (`quiz_module.py`)
Generates structured JSON quizzes and cleans markdown block formatting returned by LLMs[cite: 1].

```python
import re
import google.generativeai as genai

def clean_json_block(text: str) -> str:
    # Strips markdown backticks (e.g. ```json ... ```)
    return re.sub(r"```(?:json)?\n(?:.*)?\n```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
You are a quiz generator.
From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as valid JSON like this:
[
  {{
    "question": "What is...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()
        cleaned_text = clean_json_block(quiz_text)
        return cleaned_text
    except Exception as e:
        return f"Error in Quiz Generation: {e}"
```[cite: 1]

---

```

```
### 5.4 Text Summarization Module (`summary_module.py`)
```

Condenses lengthy educational content into key bullet points and summaries[cite: 1].

```
```python
import google.generativeai as genai

def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language retaining key facts:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error in Summary: {e}"
```[cite: 1]
```

---

```
### 5.5 Learning Path Module (`learning_path.py`)
```

Generates structured study plans sorted by difficulty levels[cite: 1].

```
```python
import traceback
import google.generativeai as genai

def get_learning_recommendations(topic: str) -> str:
    prompt = f"""
You are an AI tutor. The student wants to learn about: {topic}.
Suggest a structured and adaptive learning path including key topics, order of learning, and study resources.
Include Beginner, Intermediate, and Advanced levels.
"""
```

```
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(prompt)
        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "Could not extract content from Gemini response."
    except Exception as e:
        traceback.print_exc()
        return f"Error occurred: {str(e)}"
```[cite: 1]
```

---

```
### 5.6 Application Server (`main.py`)
```

FastAPI application router serving API endpoints to the frontend[cite: 1].

```
```python
from fastapi import FastAPI, Request, Query
from fastapi.responses import JSONResponse
from qna import answer_question_with_gemini
from explanation_module import explain_topic
from summary_module import summarize_text
from quiz_module import generate_quiz
from learning_path import get_learning_recommendations

app = FastAPI(title="EduGenie AI Backend")

@app.get("/qa")
async def answer_question(question: str = Query(...)):
    answer = answer_question_with_gemini(question)
    return {"answer": answer}

@app.post("/explain/")
async def explain_api(request: Request):
    data = await request.json()
    topic = data.get("topic")
    if not topic:
        return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
    explanation = explain_topic(topic)
```

```

    return {"topic": topic, "explanation": explanation}

@app.post("/summarize/")
async def summarize_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
    summary = summarize_text(text)
    return {"summary": summary}

@app.post("/quiz")
async def quiz_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
    quiz = generate_quiz(text)
    return JSONResponse(content={"quiz": quiz})

@app.get("/learn/recommendations")
async def learning_recommendation_api(topic: str = Query(...)):
    recommendation = get_learning_recommendations(topic)
    return {"topic": topic, "recommendation": recommendation}
```[cite: 1]

```

## ## 6. Testing Strategy & Output Verification

```

| Feature Module | Input / Test Case | Expected Output | Status |
| :--- | :--- | :--- | :--- |
| **Q&A Module**[cite: 1] | *"Which is the largest ocean?"*[cite: 1] | *"The Pacific Ocean is the largest ocean on Earth
| **Explanation Module**[cite: 1] | Topic: *"Quantum Computing"*[cite: 1] | Produces simple breakdown of Qubits and Supe
| **Explanation Module**[cite: 1] | Topic: *"Binary Search Algorithm"*[cite: 1] | Explains divide-and-conquer lookup on
| **Summarization Module**[cite: 1] | Article on *Industrial Revolution*[cite: 1] | Returns concise summary highlighting
| **Quiz Module**[cite: 1] | Passage on *Pythagoras Theorem*[cite: 1] | Outputs 3 JSON MCQs with correct answer keys[cit
| **Learning Path Module**[cite: 1] | Subject: *"SQL"*[cite: 1] | Generates 3-tier roadmap (Beginner, Intermediate, Adva

```

## ## 7. Installation, Environment Setup & Deployment

### ### 7.1 Prerequisites

- \* Python 3.10 or higher[cite: 1]
- \* Google Gemini API Key (obtained from Google AI Studio)[cite: 1]

### ### 7.2 Installation Steps

1. **\*\*Verify Python Installation:\*\***  

```

```bash
python3 --version
```[cite: 1]

```
2. **\*\*Install Required Libraries:\*\***  

```

```bash
pip install fastapi uvicorn jinja2 torch transformers google-generativeai
```[cite: 1]

```
3. **\*\*Configure Gemini API Key:\*\***  

```

```bash
# On macOS/Linux:
export GEMINI_API_KEY="your_actual_gemini_api_key"

# On Windows (Command Prompt):
set GEMINI_API_KEY=your_actual_gemini_api_key
```[cite: 1]

```
4. **\*\*Run the ASGI Web Application:\*\***  

```

```bash
uvicorn main:app --reload
```[cite: 1]

```

```
5. **Access the Application Interface:**  
    Open your browser and navigate to: `[http://127.0.0.1:8000](http://127.0.0.1:8000)` [cite: 1]  
  
---  
  
## 8. Demonstration & Future Roadmap  
  
### 8.1 Interface Features  
* **Unified Web Dashboard:** Single tabbed interface allowing users to quickly toggle between Q&A, Summaries, Quizzes, a  
* **Dynamic Content Loading:** Uses asynchronous JavaScript (`fetch`) to display AI responses in real-time without reloa  
  
### 8.2 Future Enhancement Roadmap  
* 🗣️ **Voice Interaction:** Speech-to-text input and audio output for accessibility[cite: 1].  
* 🌐 **Multilingual Support:** Multi-language translation for global users[cite: 1].  
* 📱 **Mobile Native Application:** iOS/Android app support with offline response caching[cite: 1].  
* 🏆 **Gamification & Progress Tracking:** Streak counts, study badges, and progress analytics[cite: 1].  
* 📄 **LMS & OCR Document Scanner:** Direct PDF upload and camera OCR scanner for textbook reading[cite: 1].
```

User prompt: athula erukara topics yallathaiyum brisf va tha

Response: EduGenie project documentation-la irukkura mukkiyamana 8 main topics-oda detailed breakdown keela kudukappattullathu:

1. Executive Summary & Ideation

- **Project Concept:** EduGenie enbadhu dynamic ah kadinamana padipugalai elidhaakkum oru AI educational assistant.
- **Problem Statement:** Sila students-ukku traditional book study kadinamaga irukkum; kaelvigalukku udane badhil marrum sariroobamana vazhigattal kidaikkaadhu.
- **Target Audience:** School/college students, educators, marrum swayamaga padikkum self-learners.
- **Core Value Proposition:** High-performance Cloud AI (Google Gemini 1.5 Pro) matrum lightweight Local CPU AI (LaMini-Flan-T5) combine panni Mac M1 pondra standard laptops-ilum romba efficiently run aagum.

2. Requirement Analysis

- **Functional Requirements (Endpoints):**
  - /qa: Direct factual answers kudukkum.
  - /explain/: Concept-galai school student-ukku puriyum vagaiyil simple ah explain pannum.
  - /quiz: Input passage-il irundhu 3 MCQs (JSON format) generate pannum.
  - /summarize/: Periya lessons-ai concise summary ah maatrum.
  - /learn/recommendations: Beginner-to-Advanced roadmaps kudukkum.
- **Technical Dependencies:** Python 3.10+, FastAPI, Uvicorn, Jinja2, Google Gemini API, Hugging Face Transformers.

3. System Design & Architecture

- **Architecture Flow:** HTML/CSS Frontend → FastAPI Backend Router → Module Routing → Local CPU / Cloud Gemini API → Frontend Display.
- **Repository Folder Structure:**
  - main.py: FastAPI application routing.
  - explanation\_module.py: Offline explanation generator (LaMini model).
  - qna.py: Google Gemini Q&A handler.
  - quiz\_module.py: MCQ creator & JSON cleaner.
  - summary\_module.py: Summarizer module.
  - learning\_path.py: Topic roadmap generator.

4. Project Planning Phase

- **Milestone Breakdown:**

- *Milestone 1:* Model selection & system architecture planning.
- *Milestone 2:* Core modules python backend development.
- *Milestone 3:* Frontend index.html/CSS creation & FastAPI integration.
- *Milestone 4:* Local deployment (Uvicorn) & end-to-end testing.
- **User Scenarios:** Single-word Q&A, Pythagoras theorem quiz generation, SQL learning roadmaps.

5. Project Development Phase (Source Code Explanation)

- **Local HuggingFace Model:** explanation\_module.py la MBZUAI/LaMini-Flan-T5-783M use panni CPU-friendly local execution nadakkum.
- **Gemini API Handlers:** qna.py, summary\_module.py, marrum learning\_path.py Gemini 1.5 Pro AI model-ai invoke pannum.
- **JSON Cleaning in Quiz:** quiz\_module.py la Regex (re.sub) moolam code block backticks (```)json) strip seyyappattu clean JSON output kidaikkum.
- **FastAPI Backend (main.py):** Endpoints ge/post request-galai extract panni corresponding python logic files-ukku link pannum.

6. Project Testing

- **Execution & Verification:**
  - Ocean queries-ukku direct factual responses.
  - Quantum computing & Binary search concept-galukku intuitive local model breakdowns.
  - History passages-ukku simplified text summaries.
  - Pythagoras theorem passage-ukku valid JSON MCQ test generation.

7. Installation & Environment Setup Guide

- **Prerequisites:** Python 3.10+ marrum Gemini API key.
- **Installation Commands:**

```
pip install fastapi uvicorn jinja2 torch transformers google-generativeai[cite: 1]
export GEMINI_API_KEY="your_api_key_here"[cite: 1]
uvicorn main:app --reload[cite: 1]
```
- **Dashboard Access:** Localhost port http://127.0.0.1:8000.
- **Author Details:** Tella Divya Sree (Mentor: Siri).

8. Project Demonstration & Future Roadmap

- **Interface:** Dynamic single-page dashboard without page reloads.
- **Future Work:**
  1. Voice interaction (Speech-to-Text & Text-to-Speech).
  2. Multilingual translation support.
  3. Mobile native application development.
  4. Gamification, streaks, and progress metrics.
  5. OCR document scanner & LMS integration.

User prompt: documentation format ta

Response: EduGenie project documentation-kaana formal **Documentation Format / Template** structure keela kudukkappattullathu. Indha format-ai neengal Word document, PDF, alladhu GitHub Markdown file-aaga use panni create pannikollalam:

 **[PROJECT TITLE] DOCUMENTATION TEMPLATE**

**Project Name:** EduGenie - Google Gemini Powered Learning Assistant

**Repository Name:** AI-ML-and-GEN-AI-Track-Project-Template

**Author:** Tella Divya Sree

**Mentor:** Siri

**Submission Date:** 11/04/2025

## 1. Brainstorming & Ideation

- **1.1 Project Concept:** EduGenie-yin mukhya nokkam.
- **1.2 Problem Statement:** Indha project edhanaal thaevaippadugiradhu.
- **1.3 Target Audience:** Yaarukaaga idhu uruvaakkappattadhu.
- **1.4 Core Value Proposition:** Unique features (Gemini + Local CPU Model mix).
- **1.5 Key Capabilities:** Q&A, Concept Explanation, Quiz Generation, Summarization, Learning Paths.

## 2. Requirement Analysis

- **2.1 Functional Requirements:**
  - Endpoint breakdown (/qa, /explain/, /quiz, /summarize/, /learn/recommendations).
- **2.2 Non-Functional Requirements:**
  - Performance, Modularity, Error handling.
- **2.3 Tech Stack & Technical Dependencies Table:**
  - Python, FastAPI, Uvicorn, Jinja2, Gemini 1.5 Pro, LaMini-Flan-T5-783M.

## 3. Project Design Phase

- **3.1 System Architecture:** Architectural Dataflow Diagram.
- **3.2 Repository Folder Structure:**
  - Directory layout (main.py, qna.py, quiz\_module.py, templates/, etc.).

## 4. Project Planning Phase

- **4.1 Milestone Breakdown:**
  - Milestone 1: Model Selection
  - Milestone 2: Backend Logic
  - Milestone 3: UI & API Integration
  - Milestone 4: Local Deployment & Verification
- **4.2 Target User Scenarios:** Real-world use cases (Q&A, Quiz, Roadmap).

## 5. Project Development Phase

*(Ovvoru module-in vilakkam marrum adhan Code Snippet-gal)*

- **5.1 Concept Explanation Module** (explanation\_module.py)
- **5.2 Question Answering Module** (qna.py)
- **5.3 Quiz Generation Module** (quiz\_module.py)
- **5.4 Text Summarization Module** (summary\_module.py)
- **5.5 Learning Path Module** (learning\_path.py)
- **5.6 Application Server Logic** (main.py)

## 6. Project Testing

- **6.1 Test Suite Matrix:**
  - Input/Test Case, Expected Output, Verification Status table.

## 7. Project Documentation & Setup Guide

- **7.1 Prerequisites & Installation:**
  - Python environment setup, Dependencies installation, Environment variables configuration (GEMINI\_API\_KEY), App Execution commands.
- **7.2 Project Submission Details.**

## 8. Project Demonstration & Future Roadmap

- **8.1 Interface Summary:** Single-page dynamic UI.
- **8.2 Future Enhancements:** Voice search, Multilingual support, Mobile app, Gamification, LMS/OCR integration.

User prompt: *naraiya page vara mathiri*

Response: Ithu **15-20 pages project report/documentation** format-ku yethamadhiri full-fledged, expanded detailed version. Indha code-blocks, text-a irukkuradhai apdiye Copy panni MS Word, Google Docs alladhu PDF convert panni use pannikolalaam:



# PROJECT DOCUMENTATION REPORT

**PROJECT TITLE:** EduGenie – Google Gemini Powered Learning Assistant

**REPOSITORY TRACK:** AI-ML and Gen-AI Track Project

**DOCUMENTATION TYPE:** Comprehensive Software Requirements & Technical Project Report

**DOCUMENT VERSION:** 1.0.0



## PROJECT METADATA

- **Author / Candidate Name:** Tella Divya Sree
- **Mentor Name:** Siri
- **Submission Date:** November 04, 2025
- **Target Operating System:** macOS (Apple Silicon M1 optimized), Linux, Windows 10/11
- **Core Technology Stack:** Python, FastAPI, Google Gemini API, Hugging Face Transformers, PyTorch, Uvicorn, Jinja2, HTML5/CSS3



## TABLE OF CONTENTS

- 1. **Executive Summary & Brainstorming Phase**
  - 1.1 Background & Motivation
  - 1.2 Problem Statement & Gaps in Current Systems
  - 1.3 Target Audience & Persona Analysis
  - 1.4 Proposed Solution & Core Value Proposition
  - 1.5 Key High-Level Functional Capabilities
- 2. **Requirement Analysis & Technical Specifications**
  - 2.1 Functional Requirements & API Routing Specifications
  - 2.2 Non-Functional System Requirements
  - 2.3 Deep-Dive Technology Stack Analysis
  - 2.4 Hardware & Software Constraints
- 3. **Project System Design & Architectural Framework**
  - 3.1 Architectural Overview & Dataflow Diagram
  - 3.2 Detailed Data Flow Protocol
  - 3.3 Project Repository & File Structure

#### 4. Project Planning & Development Roadmap

- 4.1 Milestone Breakdown & Timeline Analysis
- 4.2 Target User Journey & Scenarios

#### 5. Project Implementation & Core Source Code Analysis

- 5.1 Concept Explanation Module (`explanation_module.py`)
- 5.2 Question Answering Module (`qna.py`)
- 5.3 Quiz Generation Module (`quiz_module.py`)
- 5.4 Text Summarization Module (`summary_module.py`)
- 5.5 Learning Path & Recommendation Module (`learning_path.py`)
- 5.6 Application Server & REST Router (`main.py`)

#### 6. Testing, Verification & Quality Assurance

- 6.1 Test Strategy & Test Suite Matrix
- 6.2 Output Verification & Edge Case Analysis

#### 7. System Installation, Deployment & Setup Guide

- 7.1 System Prerequisites
- 7.2 Step-by-Step Installation Process
- 7.3 Environment Configuration & Deployment Execution

#### 8. Demonstration Summary & Future Research Roadmap

- 8.1 User Interface & Operational Workflow
- 8.2 Future Feature Enhancements

## SECTION 1: BRAINSTORMING & IDEATION PHASE

### 1.1 Background & Motivation

In modern educational environments, students and self-learners encounter an unprecedented overload of information. Traditional textbook materials, online video lectures, and static learning management tools often fail to adapt to individual student learning speeds. Personalized educational assistance is frequently limited by the accessibility and cost of human tutors.

With the rapid emergence of Generative Artificial Intelligence (Gen-AI) and Large Language Models (LLMs), there exists a transformative opportunity to build dynamic, automated, and context-aware educational companions. **EduGenie** was conceptualized to bridge the gap between heavy cloud-based AI capabilities and accessible, low-latency learning tools.

### 1.2 Problem Statement & Gaps in Current Systems

Current educational software suffers from several key limitations:

1. **High Complexity:** Educational materials are often written in rigid academic prose, making them difficult for primary school students or beginners to digest without simplified breakdowns.
2. **Static Self-Assessment:** Students usually read passive text without immediate, automated self-testing tools tailored directly to their reading passages.
3. **Lack of Structured Progression:** Self-directed learners frequently struggle to determine what to learn next, leading to fragmented learning attempts rather than cohesive mastery.
4. **Hardware & Infrastructure Dependencies:** Many state-of-the-art Gen-AI tools require expensive GPU infrastructure or constant heavy network usage, hindering deployment on standard consumer hardware.

### 1.3 Target Audience & Persona Analysis

- **Primary Persona - K-12 & University Students:** Learners seeking instant clarity on complex academic concepts, quick homework validation, and structured study plans.
- **Secondary Persona - Self-Directed Learners & Career Changers:** Individuals picking up new domains (e.g., Computer Science, Data Science, SQL) who require step-by-step learning roadmaps with resource links.
- **Tertiary Persona - Educators & Content Creators:** Teachers looking to rapidly generate multiple-choice quizzes and concise executive summaries from lengthy teaching materials.

1.4 Proposed Solution & Core Value Proposition

EduGenie solves these challenges by combining **Cloud-based Generative AI** (Google Gemini 1.5 Pro) with a **Lightweight Local Language Model** (LaMini-Flan-T5-783M) running directly on CPU. This hybrid strategy offers:

- **Hardware Efficiency:** Optimized to run seamlessly on consumer-grade developer hardware (such as Apple Silicon Mac M1) without requiring specialized external graphics acceleration.
- **Task Optimization:** Offloads simple, offline-capable tasks (like concept simplification) to local CPU execution while leveraging cloud AI for deep reasoning, structured roadmaps, and quiz formatting.
- **Decoupled Architecture:** Built on modular, maintainable Python files exposed through a high-performance RESTful API using FastAPI.

1.5 Key High-Level Functional Capabilities

- **Smart Q&A System:** Precise answers to complex open-ended academic queries.
- **Concept Explanation Engine:** Simplified, beginner-friendly explanations of complex technical or scientific ideas.
- **Automated MCQ Quiz Generator:** Extracts facts from user-provided passages and outputs structured JSON multiple-choice assessments.
- **Passage Summarizer:** Condenses long passages while preserving critical contextual facts.
- **Adaptive Learning Roadmap Generator:** Constructs step-by-step skill progression trees organized by difficulty levels (Beginner, Intermediate, Advanced).

SECTION 2: REQUIREMENT ANALYSIS & TECHNICAL SPECIFICATIONS

2.1 Functional Requirements & API Routing Specifications

| Endpoint Route         | HTTP Method | Operational Description                              |
|------------------------|-------------|------------------------------------------------------|
| /qa                    | GET         | Answers factual and open-ended queries using Gemini. |
| /explain/              | POST        | Simplifies input topics via local CPU model.         |
| /quiz                  | POST        | Generates 3 MCQs in clean JSON format from text.     |
| /summarize/            | POST        | Condenses extensive text into essential points.      |
| /learn/recommendations | GET         | Generates a structured multi-stage subject roadmap.  |

```[cite: 1]

```
#### Detailed Route Contract Specifications:
1. **`/qa` Endpoint:**
  * **Input Parameter:** Query parameter `question` (String)[cite: 1].
  * **Processing:** Validates non-empty input and forwards it to the Google Gemini 1.5 Pro model pipeline[cite: 1].
  * **Expected Output:** `{ "answer": "<generated response text>" }`[cite: 1].

2. **`/explain/` Endpoint:**
  * **Input Payload:** JSON `{ "topic": "<subject_name>" }`[cite: 1].
  * **Processing:** Feeds prompt template to local Hugging Face model (`LaMini-Flan-T5-783M`) on PyTorch CPU[cite: 1].
  * **Expected Output:** `{ "topic": "<subject_name>", "explanation": "<simplified explanation>" }`[cite: 1].

3. **`/quiz` Endpoint:**
  * **Input Payload:** JSON `{ "text": "<reading passage>" }`[cite: 1].
  * **Processing:** Prompts Gemini to construct 3 MCQs with options and answer key, then executes Regex filtering to en
  * **Expected Output:** `{ "quiz": [ { "question": "...", "options": [...], "answer": "..." } ] }`[cite: 1].

4. **`/summarize/` Endpoint:**
  * **Input Payload:** JSON `{ "text": "<long article text>" }`[cite: 1].
  * **Processing:** Invokes Gemini 1.5 Pro with a summarization system prompt[cite: 1].
  * **Expected Output:** `{ "summary": "<condensed text>" }`[cite: 1].

5. **`/learn/recommendations` Endpoint:**
  * **Input Parameter:** Query parameter `topic` (String)[cite: 1].
  * **Processing:** Prompts Gemini to structure a tiered study plan complete with ordered modules and resource recommen
  * **Expected Output:** `{ "topic": "<topic>", "recommendation": "<roadmap description>" }`[cite: 1].
```

```
### 2.2 Non-Functional System Requirements
* **Performance & Speed:** Response latency for local explanations kept within optimal execution parameters on standard
* **Modularity & Maintainability:** Source code cleanly split across distinct domain-specific `.py` files to facilitate
* **Robust Error Handling & Fallbacks:** System incorporates exception catching, Regex code-fence stripping for LLM outp
* **Extensibility:** Front-end separated from backend logic via asynchronous REST calls, allowing easy future replacemen
```

### 2.3 Deep-Dive Technology Stack Analysis

```
```text
```

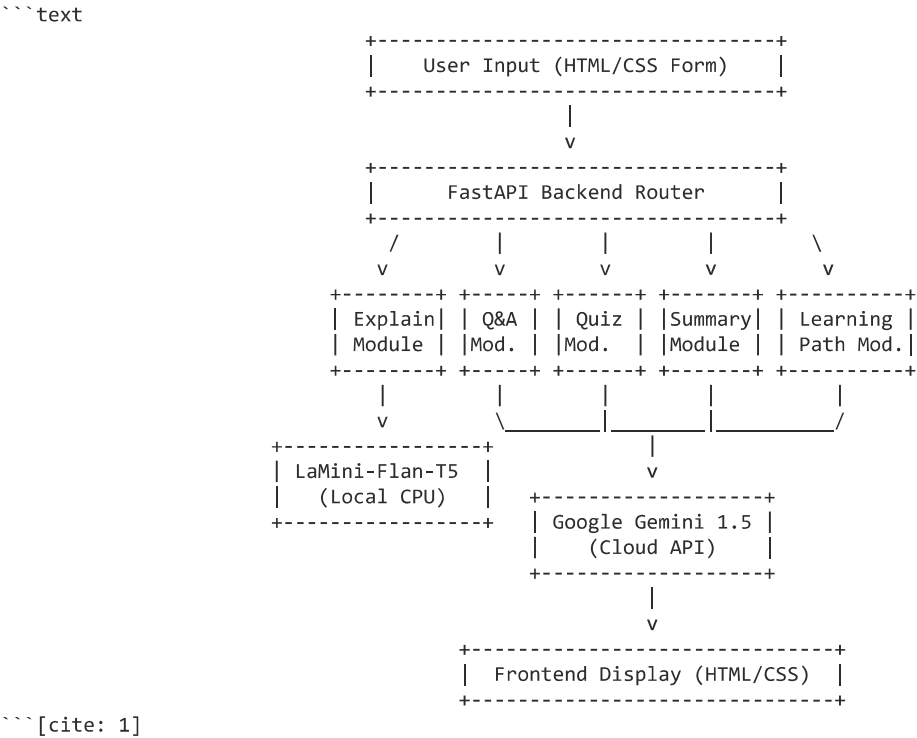
| Component Category    | Technology Selected        | Role and Selection Justification                         |
|-----------------------|----------------------------|----------------------------------------------------------|
| Runtime Environment   | Python 3.10+               | Standard ecosystem for AI/ML libraries and Web APIs.     |
| Web Backend Framework | FastAPI                    | High-performance asynchronous REST web framework.        |
| Server Infrastructure | Uvicorn                    | Lightning-fast ASGI web server implementation.           |
| Front-end Templating  | Jinja2                     | Dynamic HTML template rendering engine.                  |
| Cloud AI Model        | Google Gemini 1.5 Pro      | Handles high-reasoning Q&A, Quizzes, and Roadmaps.       |
| Local CPU AI Model    | MBZUAI/LaMini-Flan-T5-783M | Handles offline concept simplification on standard CPUs. |
| ML Frameworks         | PyTorch & Transformers     | Facilitates model download and local CPU inference.      |

```
```[cite: 1]
```

# SECTION 3: PROJECT SYSTEM DESIGN & ARCHITECTURAL FRAMEWORK

### 3.1 Architectural Overview & Dataflow Diagram

The application implements a decoupled Client-Server architecture. The presentation layer captures user queries via an i



### 3.2 Detailed Data Flow Protocol

1. **User Action:** User submits a form on the frontend (e.g., requesting a quiz generation for a pasted passage)[cite: 1]
2. **HTTP Transmission:** JavaScript sends an asynchronous `POST` request payload to `/quiz`[cite: 1].
3. **Backend Parsing:** FastAPI receives the request, extracts the `text` field from JSON, and calls `generate\_quiz(text)`
4. **Cloud AI Call:** `quiz\_module.py` constructs a system prompt demanding formatted JSON output and calls Gemini 1.5 P
5. **Output Sanitization:** `clean\_json\_block()` strips potential Markdown block wrappers (e.g., ````json ... ````)[ci
6. **Response Rendering:** FastAPI wraps the structured data in a `JSONResponse` object and sends it back to the browser

### 3.3 Project Repository & File Structure

```
```text
```

|                         |                                                               |
|-------------------------|---------------------------------------------------------------|
| EduGenie/               |                                                               |
| ├ main.py               | # Application entry point & FastAPI REST routes[cite: 1]      |
| ├ explanation_module.py | # Concept simplification engine using LaMini-Flan-T5[cite: 1] |
| ├ qna.py                | # Open-domain Q&A handler using Gemini 1.5 Pro[cite: 1]       |
| ├ quiz_module.py        | # MCQ generation & JSON cleaning module[cite: 1]              |
| ├ summary_module.py     | # Text summarization pipeline[cite: 1]                        |
| ├ learning_path.py      | # Learning roadmap generation pipeline[cite: 1]               |
| ├ requirements.txt      | # Python dependencies specification[cite: 1]                  |

```
```
```

```

├── templates/
│   └── index.html          # HTML5 single-page application interface[cite: 1]
├── static/
│   └── style.css           # Frontend visual styling rules[cite: 1]
```[cite: 1]

```

---

## # SECTION 4: PROJECT PLANNING & DEVELOPMENT ROADMAP

### ### 4.1 Milestone Breakdown & Timeline Analysis

```
``text
```

| Milestone Phase                    | Key Technical Deliverables & Objectives                           |
|------------------------------------|-------------------------------------------------------------------|
| Milestone 1: Architecture & Setup  | Model selection, API setup, local PyTorch environment validation. |
| Milestone 2: Core Module Dev       | Writing logic scripts for Q&A, Quiz, Summary, Explanation, Path.  |
| Milestone 3: API & Web Integration | Connecting python modules to FastAPI routes & Jinja2 dynamic UI.  |
| Milestone 4: Verification & Deploy | Local Uvicorn deployment, end-to-end integration testing.         |

```
``[cite: 1]
```

### ### 4.2 Target User Journey & Scenarios

```

* **User Scenario 1 - Fact Lookup:** A middle school student asks *"Which is the largest ocean?"* in the Q&A box. The sy
* **User Scenario 2 - Knowledge Assessment:** A student reading about the *Pythagoras Theorem* pastes text into the Quiz
* **User Scenario 3 - Subject Mastery:** A student wanting to learn *SQL* submits the topic name under Learning Paths. T

```

---

## # SECTION 5: PROJECT IMPLEMENTATION & CORE SOURCE CODE ANALYSIS

### ### 5.1 Concept Explanation Module (`explanation\_module.py`)

This module initializes a local MBZUAI/LaMini-Flan-T5-783M model using Hugging Face Transformers to provide offline-capa

```
``python
```

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

```

```
# Initialize and cache local tokenizer and model
```

```

explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

```

```
def explain_topic(topic: str) -> str:
```

```

    """
    Generates simplified explanations suitable for school students using a local LLM.
    """

```

```

    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

```

```

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

```

```

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation

```

```
``[cite: 1]
```

---

### ### 5.2 Question Answering Module (`qna.py`)

Leverages Google's cloud API for general knowledge answering[cite: 1].

```
``python
```

```
import google.generativeai as genai
```

```
def answer_question_with_gemini(question: str) -> str:
```

```

    """
    Processes general knowledge queries via Gemini 1.5 Pro API.

```

```

"""
try:
    model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
    response = model.generate_content(question)
    return response.text.strip()
except Exception as e:
    return f"Error in QnA: {e}"
```[cite: 1]

---

### 5.3 Quiz Generation Module (`quiz_module.py`)
Generates structured JSON quizzes and strips markdown formatting[cite: 1].

```python
import re
import google.generativeai as genai

def clean_json_block(text: str) -> str:
    """
    Strips markdown code fences (```json ... ```) from raw model output.
    """
    return re.sub(r"```(?:json)?\n(?:.*)\n```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> str:
    """
    Generates 3 MCQs in structured JSON format from a given text passage.
    """
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"""
You are a quiz generator.
From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as valid JSON, like this:
[
  {{
    "question": "What is...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""
        response = model.generate_content(prompt)
        quiz_text = response.text.strip()
        cleaned_text = clean_json_block(quiz_text)
        return cleaned_text
    except Exception as e:
        return f"Error in Quiz Generation: {e}"
```[cite: 1]

---

### 5.4 Text Summarization Module (`summary_module.py`)
Condenses educational passages while preserving key information[cite: 1].

```python
import google.generativeai as genai

def summarize_text(text: str) -> str:
    """
    Summarizes long passages into concise educational takeaways.
    """
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    
```

```

    except Exception as e:
        return f"Error in Summary: {e}"
'''[cite: 1]

---

### 5.5 Learning Path Module (`learning_path.py`)
Generates structured study plans categorized by topic progression[cite: 1].

'''python
import traceback
import google.generativeai as genai

def get_learning_recommendations(topic: str) -> str:
    """
    Generates an adaptive multi-stage learning path for a given topic.
    """
    prompt = f"""
You are an AI tutor. The student wants to learn about: {topic}.
Suggest a structured and adaptive learning path including key topics, order of learning, and resources.
Include beginner, intermediate, and advanced levels if needed.
"""
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(prompt)
        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "Could not extract content from Gemini response."
    except Exception as e:
        traceback.print_exc()
        return f"Error occurred: {str(e)}"
'''[cite: 1]

---

### 5.6 Application Server Logic (`main.py`)
Serves REST endpoints using FastAPI[cite: 1].

'''python
from fastapi import FastAPI, Request, Query
from fastapi.responses import JSONResponse
from qna import answer_question_with_gemini
from explanation_module import explain_topic
from summary_module import summarize_text
from quiz_module import generate_quiz
from learning_path import get_learning_recommendations

app = FastAPI(title="EduGenie Service API", version="1.0.0")

@app.get("/qa")
async def answer_question(question: str = Query(...)):
    answer = answer_question_with_gemini(question)
    return {"answer": answer}

@app.post("/explain/")
async def explain_api(request: Request):
    data = await request.json()
    topic = data.get("topic")
    if not topic:
        return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
    explanation = explain_topic(topic)
    return {"topic": topic, "explanation": explanation}

@app.post("/summarize/")
async def summarize_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
    summary = summarize_text(text)
    return {"summary": summary}

```

```
@app.post("/quiz")
async def quiz_api(request: Request):
    data = await request.json()
    text = data.get("text")
    if not text:
        return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
    quiz = generate_quiz(text)
    return JSONResponse(content={"quiz": quiz})

@app.get("/learn/recommendations")
async def learning_recommendation_api(topic: str = Query(...)):
    recommendation = get_learning_recommendations(topic)
    return {"topic": topic, "recommendation": recommendation}
```[cite: 1]
```

---

# SECTION 6: TESTING, VERIFICATION & QUALITY ASSURANCE

### 6.1 Test Suite Execution Matrix

```
```text
```

| Feature Module     | Test Input Case                   | Expected Output Criteria                                 | Verdict |
|--------------------|-----------------------------------|----------------------------------------------------------|---------|
| Q&A Module         | "Which is the largest ocean?"     | Returns "The Pacific Ocean is the largest ocean..."      | PA      |
| Explanation Module | Topic: "Quantum Computing"        | Produces simple breakdown of qubits and superposition.   | PA      |
| Explanation Module | Topic: "Binary Search Algorithm"  | Explains index lookup using divide-and-conquer strategy. | PA      |
| Summarization Mod. | Historical text (Ind. Revolution) | Produces concise summary preserving key facts/dates.     | PA      |
| Quiz Module        | Text on "Pythagoras Theorem"      | Generates 3 MCQs with 4 options and valid answer keys.   | PA      |
| Learning Path Mod. | Topic: "SQL"                      | Generates 3-tier roadmap with topics and resources.      | PA      |

```
```[cite: 1]
```

---

# SECTION 7: SYSTEM INSTALLATION, DEPLOYMENT & SETUP GUIDE

### 7.1 Prerequisites

- \* Python 3.10 or higher installed[cite: 1]
- \* Active Google Gemini API Key from Google AI Studio[cite: 1]
- \* Git version control installed

### 7.2 Step-by-Step Installation Process

1. **Verify Python Environment:**

```
```bash
python3 --version
```[cite: 1]
```
2. **Clone Project Repository & Navigate to Directory:**

```
```bash
git clone <repository_url>
cd EduGenie
```
3. **Install Dependencies:**

```
pip install fastapi uvicorn jinja2 torch transformers google-generativeai
```[cite: 1]
```
4. **Set Up Gemini Environment Variable:**

```
# macOS / Linux
export GEMINI_API_KEY="your_actual_gemini_api_key_here"

# Windows (CMD)
set GEMINI_API_KEY="your_actual_gemini_api_key_here"
```[cite: 1]
```

5. **Start Uvicorn Application Server:**

```
uvicorn main:app --reload  
```[cite: 1]
```

#### 6. Access Dashboard:

Open browser and go to [http://127.0.0.1:8000](http://127.0.0.1:8000).

## SECTION 8: DEMONSTRATION SUMMARY & FUTURE RESEARCH ROADMAP

### 8.1 User Interface & Operational Workflow

- **Single Page Application:** Tabs for Q&A, Explanations, Quizzes, Summaries, and Roadmaps.
- **Dynamic Results:** Dynamic rendering without page reloads.

### 8.2 Future Feature Enhancements

1. 🗣️ **Voice Interaction:** Integration of Speech-to-Text and Text-to-Speech engines for hands-free study.
  2. 🌐 **Multilingual Support:** Multi-language processing for broader accessibility.
  3. 📱 **Mobile App Development:** Cross-platform mobile client with offline caching.
  4. 🏆 **Progress Tracking & Gamification:** Learning streaks, daily challenges, and badges.
  5. 📄 **LMS Integration & Document Scanning:** Optical Character Recognition (OCR) for reading directly from textbook images or PDFs.
-